Cooperative Data Sensing and Computation Offloading in UAV-Assisted Crowdsensing With Multi-Agent Deep Reinforcement Learning

Ting Cai, *Graduate Student Member, IEEE*, Zhihua Yang, Yufei Chen, Wuhui Chen, *Member, IEEE*, Zibin Zheng, *Senior Member, IEEE*, Yang Yu, *Member, IEEE*, and Hong-Ning Dai, *Senior Member, IEEE*

***Abstract*—Unmanned aerial vehicles (UAVs) can be leveragedin mobile crowdsensing (MCS) to conduct sensing tasks at remote or rural areas through computation offloading and data sensing. Nonetheless, both computation offloading and data sensing have been separately investigated in most existing studies. In this paper, we propose a novel cooperative data sensing and computation offloading scheme for the UAV-assisted MCS system with an aim to maximize the overall system utility. First, a multi-objective function is formulated to evaluate the system utility by jointly considering flight direction, flight distance, task offloading proportion, and server offload selection for each UAV. Then, the problem is modeled as a partially observable Markov decision process and a multi-agent actor-critic algorithm framework is proposed to train the strategy network for UAVs. Due to high delay and energy costs caused by communications among multiple agents, we train a centralized critic network to model other agents and to seek equilibrium among all UAVs rather than adopting the explicit channel for information exchange. Furthermore, we introduce attention mechanism to enhance the convergence performance in model training phases. Finally, experimental results demonstrate the effectiveness and applicability of our scheme. Compared with baselines, our algorithm shows significant advantages in convergence performance, system utility, task costs, and task completion rate.**

***Index Terms*—Mobile crowdsensing (MCS), unmanned aerial vehicle (UAV), data sensing, computation offloading, deep reinforcement learning (DRL).**

Manuscript received 27 May 2021; revised 4 September 2021; accepted 17 October 2021. Date of publication 21 October 2021; date of current version 9 September 2022. The work was supported in part by the Key-Area Research and Development Program of Guangdong Province under Grant 2019B020214006, in part by the National Natural Science Foundation of China under Grants 62032025 and 61802450, in part by the National Key Research and Development Program of China under Grant 2020YFB1707603, in part by NSFC-Guangdong Joint Fund Project under Grant U20A6003, in part by the Program for Guangdong Introducing Innovative and Entrepreneurial Teams under Grant 2017ZT07X355, in part by Pearl River Talent Recruitment Program under Grant 2019QN01X130, in part by the Science and Technology Research Program of Chongqing Municipal Education Commission under Grant KJZD-K201802401, and in part by Macao Science and Technology Development Fundunder Macao Funding Scheme for Key R&D Projects under Grant 0025/2019/AKP. Recommended for acceptance by Dr. Zehui Xiong. *(Corresponding author: Wuhui Chen.)*

Ting Cai, Zhihua Yang, Yufei Chen, Wuhui Chen, Zibin Zheng, and Yang Yu are with the School of Computer Science and Engineering, National Engineering Research Center of Digital Life, Sun Yat-sen University, Guangzhou 510006, China (e-mail: cait9@mail2.sysu.edu.cn; yangzhh28@mail2.sysu.edu.cn; chenyf75@mail2.sysu.edu.cn; chenwuh@mail.sysu.edu.cn; zhzibin@mail.sysu.edu.cn; yuy@mail.sysu.edu.cn).

Hong-Ning Dai is with the Department of Computing and Decision Sciences, Lingnan University, Hong Kong 999077, China (e-mail: hndai@ieee.org).

Digital Object Identifier 10.1109/TNSE.2021.3121690

I. Introduction

DATA applications have promoted a steady increase of smart city initiatives, such as disaster forecasts, smart healthcare, and traffic planning and management [1]–[3]. However, the absence of effective data sensing has brought poor data quality, thereby affecting data analytics and data utilization. To address this issue, mobile crowdsensing (MCS) is emerging as a promising technique to provide effective data sensing for various domains. MCS mainly leverages smart mobile devices (e.g., laptops, iWatch, sensors, and smartphones) to collect data [2], [4]. However, MCS confronts two major problems since MCS tasks mainly rely on data collection from human-centric or wearable devices. On the one hand, it is difficulty to conduct the data sensing tasks in rural, remote, and inaccessible areas. On the other hand, the limited computational capability and limited storage capacity of MCS devices also affect the effectiveness of data sensing tasks.

The recent advent of unmanned aerial vehicles (UAVs) brings opportunities to address the above challenges of MCS. First, UAVs that are equipped with built-in high precision sensors can fly to remote and rural area to conduction data sensing tasks [5], [6]. Meanwhile, highly flexible UAVs are equipped with infrared radar, high-definition camera, and high-processing sensor devices. These features make UAVs to perform high-quality data sensing tasks by moving in a larger scope and wider areas than ordinary human-centric mobile devices, and to have good flexibility. Moreover, UAVs equipped with onboard computing systems also have the stronger capability of data processing than wearable devices.

Despite the potential of UAVs in conducting MCS tasks, it is still challenging for UAVs to conduct computational-complex MCS tasks due to their limited power supply. To tackle this issue, computation offloading is recently introduced to help UAVs upload the appropriate amount of computational tasks to edge or cloud servers for further processing [7], [8]. However, there still exist some challenges expected to be addressed. We summarize these challenges from the following two aspects.

- Existing large-scale data sensing in MCS systems mainly consists of three phases: data collection, data storage and processing, and data uploading [9]. It is necessary to consider these three phases together to ensure

2327-4697 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on March 29,2024 at 06:52:56 UTC from IEEE Xplore. Restrictions apply.

the high-quality sensed data, which is the core of MCS. However, all these three phases involve complex computing and communication operations, e.g., data processing contains all computation-related data operations. Despite the advances in current UAV-based MCS schemes [10]–[13], most of them study the design and optimization of data sensing and computation offloading separately, thereby achieving a sub-optimal performance. Therefore, a joint design of sensing and offloading is highly expected for the UAV-assisted MCS system with an aim to improve overall system performance.

- It is time-consuming to solve the optimization problem of data sensing and computation offloading by dynamic programming or greedy algorithms due to multiple iterations [14]. Meanwhile, these algorithms also confront critical challenges when gaining instant system models during execution in a diversified and time-variant MCS environment [15]. Although deep reinforcement learning (DRL) has shown its strengths in solving these complex and dynamic issues by modeling the joint optimization problem as a Markov decision process (MDP), most single-agent DRL algorithms like deep deterministic policy gradient (DDPG) [16], [17] inevitably lead to the communication delay and the increase of computational complexity due to the increased dimension of state and action spaces due to the increased number of UAVs. Despite the advances of multi-agent DRL solutions, each agent in the UAV-assisted MCS system may only observe the local environment and receive a partially observation without considering environmental updates because of policies of other agents, which makes the learning environment non-stationary [18], [19]. Moreover, multi-agent DRL is not capable of learning the cooperative data sensing and computation offloading policies of UAVs.

To address the above limitations, we propose a novel cooperative data sensing and computation offloading scheme for UAV-assisted MCS systems enabled by an improved multi-agent DRL (called AT-MARL) algorithm. In particular, the main objective of AT-MARL is to maximize the overall system utility while considering both sensing utility and offloading utility together. More explicitly, each UAV flies to collect sensing data and then handles partial computation tasks locally while offloading other tasks to be processed in remote edge or cloud servers. We regard the UAV-assisted MCS as a multi-agent system, which is modeled as a partially observable Markov decision process (POMDP). In particular, we present an enhanced distributed DRL-based algorithm based on multi-agent actor-critic policy optimization to maximize the long-term reward of the UAV-assisted MCS systems. To speed the convergence in model training and tackle the non-stationary issues arising from training iterations of multiple UAVs, we train the centralized critic network. The learned model is then executed at each UAV in a distributed manner. To further enhance performance, we introduce the attention mechanism [20], [21], which helps each agent pay different attention to other agents when modeling the critic network. In conclusion, the main contributions of this paper are highlighted as follows:

- We study the joint problem of data sensing and computation offloading for range-sensitive and computation-intensive applications in UAV-assisted MCS. To optimize the long-term performance, we formulate the joint optimization problem as POMDP and take advantage of the multi-agent DRL to solve the problem.
- We propose the AT-MARL algorithm to improve the actor-critic framework by embedding multi-head attention units in existing network structure, which enhances the convergence performance of the critic network and accordingly accelerates the training speed of the actor network.
- We compare our scheme with other baselines through extensive simulations. Results demonstrate our significant advantages in terms of higher task completion rate, lower task costs, and the best overall system utility than other baselines. Moreover, we develop a real-world UAV testing platform to validate the effectiveness and applicability of the proposed AT-MARL algorithm.

The remainder of this paper is organized as follows. We first introduce the related work in Section II and then present the system model Section III. In Section IV, we formulate the joint problem as a POMDP. The design of the AT-MARL algorithm is given in Section V. Section VI presents simulations and experiments. Finally, we conclude this paper and give future directions in Section VII.

II. Related Work

A. UAV-Based MCS System

Due to the flexibility, mobility, and sensing ability of UAVs, many researchers have studied the UAV-based MCS system from multiple perspectives [22]–[24]. Because of the limited energy of UAVs, it is necessary to properly design a routing plan and task assignment so as to minimize the time and energy cost for participating UAVs in the MCS system. Zhou *et al.* [25] proposed a cooperative task assignment and routing plan for the MCS system with consideration of the energy cost of UAVs. Wang *et al.* [26] investigated a UAV-assisted MCS system in dynamic environments by formulating the task assignment in the stochastic environment into a dynamic matching problem. They then proposed a multiple-waitlist-based task assignment to solve the problem. Wang *et al.* [5] concentrated on the privacy-preserving problem of UAVs and proposed a federated learning framework with differential-privacy and blockchain technology for the UAV-based MCS systems. Different from the above-mentioned studies which mainly focus on the data sensing procedure in the UAV-based MCS systems, some researchers have already investigated the computation offloading in data processing of mobile edge computing (MEC) due to the limited computing power provided by UAVs [27]–[29]. Messous *et al.* [10] used the game theory to model and balance the execution time and the energy cost of UAV-based MEC systems. Cao *et al.* [11]

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on March 29,2024 at 06:52:56 UTC from IEEE Xplore. Restrictions apply.

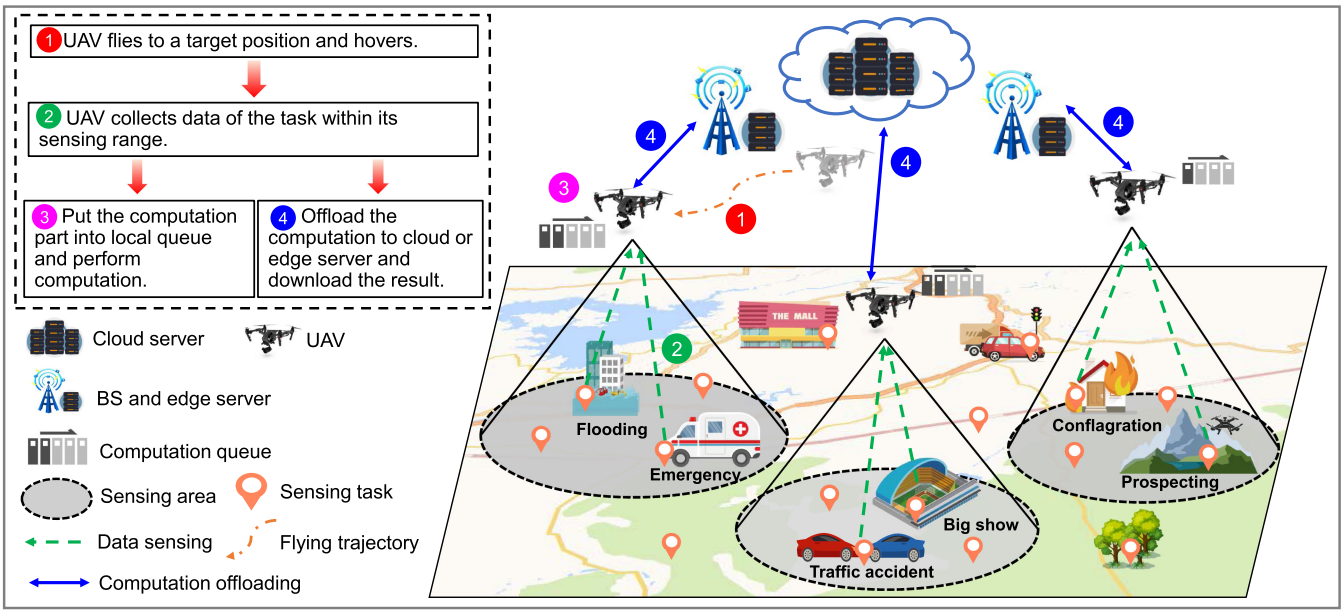


Fig. 1. Overview of the UAV-assisted MCS system model.

studied the joint optimization about the trajectory planning and the computation offloading of the UAV network served by cellular ground base stations. Mei *et al.* [12] studied the cooperative resource allocation and trajectory planning problem in UAV-based MEC systems. Zhang *et al.* [13] proposed a joint optimization algorithm to minimize the system cost based on the constraints of computation offloading and trajectory scheduling.

However, the above-mentioned researches mainly focused on either data sensing or computation offloading while the joint optimization schemes of trajectory scheduling and computation offloading in the MEC system cannot be directly applied in the MCS system because of lacking the data sensing procedure [10]–[13]. Moreover, most of these studies are required to obtain an instant system model during execution, which nevertheless is difficult to be acquired in practical applications. Therefore, these limitations motivate us to propose a unified framework to solve the cooperative data sensing and computation offloading problem without the knowledge of real-time environment modeling in a UAV-based MCS system.

B. Deep Reinforcement Learning for the MCS System

Fortunately, the recent advances of DRL provide alternative approaches on designing the MCS system without the knowledge of real-time environment [30]–[32]. In particular, Piao *et al.* [33] proposed a sequential DRL approach to address the task assignment and routing plan problem in the MCS system. Tao *et al.* [34] developed an efficient algorithm based on the double deep-Q-network to solve the task-allocation problem in the MCS system. Liu *et al.* [35] studied the multi-task-oriented vehicular crowdsensing problem and proposed a DRL-based approach to maximize the energy efficiency while evaluating the sensing task in a dynamic environment. Xu *et al.* [36] proposed a DRL-based approach for autonomously controlling the uplink transmission rate from mobile IoT devices to access stations, which can capture the variations of the uplink channels and improve the rate selections. Moreover, Zhang *et al.* [37] proposed an effective DRL-based control framework to address the problem of data sensing and charging in UAV-based MCS systems and evaluated the framework on a real dataset in Rome. However, since the problem modeling of real-world is becoming increasingly complex, it is difficult to regard the whole complicated system as a single agent for centralized controlling. The reason lies in the communication delay between the centralized server and the other devices and the overlarge dimension of the neural network under a complicated environment.

Recently, some researchers have focused on the multi-agent-DRL for intelligent MCS [38], [39]. Heydari *et al.* [40] proposed a non-cooperative multi-agent-DRL approach for solving the joint optimization problem in the MEC network. Zhang *et al.* [41] developed a multi-agent-DRL approach for MEC system, where the edge devices improved their policy independently. However, these approaches mainly focused on the independent-model learning, where all the agents learn their policy individually without any cooperation. As a result, the learning environment is getting unstable while each agent regards other mutable agents as stable agents, thereby resulting in invalidating the convergence features and degrading the system performance. Therefore, this gap motivates us to propose a collaborative multi-agent-DRL scheme for the MCS system.

III. System Model

As shown in Fig. 1, we consider a UAV-assisted MCS system that consists of a cloud server and K edge servers. We denote all the servers by a set $\mathcal{K} = \{0, 1, \ldots, K\}$, where 0 represents the cloud server and others are edge servers. All the servers provide the IoT devices participating in MCS with computation services. We assume that there is a set of tasks,

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on March 29,2024 at 06:52:56 UTC from IEEE Xplore. Restrictions apply.

i.e., $\mathcal{M} = \{1, 2, \ldots, M\}$, each of which is associated with a certain amount of data $b_j,\ j \in \mathcal{M}$ within a target area in each time slot. The execution of each task can be divided into two stages, i.e., data sensing and computation. Meanwhile, there are $\mathcal{N} = \{1, 2, \ldots, N\}$ UAVs being scheduled to collect sensing data and then perform a subset of computation tasks according to the offloading strategy. More specifically, each UAV $i \in \mathcal{N}$ flies to a certain distance $d_t^i \in [0, d_{\max}]$ with a certain direction θ_t^i firstly at a fixed height within one time slot t, where $d_{\max}$ denotes the maximum distance that UAV can move in a time slot. We assume that each UAV has a limit sensing capability represented by a sensing range r_s^i. Then, UAVs hover at the specific position and start to collect sensing data within their sensing range r_s^i.

The workflow of our system model is illustrated in Fig. 1. At the beginning of each epoch t, a set of tasks $\mathcal{M}$ locate randomly within a target area and the UAV-assisted MCS system assigns the task information to each UAV, including the coordinates of a task position, the amount of data collection, and the amount of computing tasks. Then, each UAV completes the task with a three-step process. First, UAV flies to the target position and hovers. Second, UAV collects data of the task within its sensing range. Finally, after collecting data, UAV performs computations by using a combination of local computation and computation offloading. Without loss of generality, we assume that UAVs can perform data sensing and computation simultaneously. Owing to the insufficient computation power and limited battery life, UAV needs to offload a proportional computation task to the edge or cloud servers. In detail, the tasks handled locally are first put in a local queue and are then executed on the principle of First-In-First-Out (FIFO). Regarding computation offloading, UAV uploads data to server z_t^i with a transmission rate ρ_t^i (related to the bandwidth and the number of UAVs connecting to the server z_t^i). We assume that each server is mounted with computing facilities (e.g., enough number of CPU cores) so that all the jobs can be handled concurrently. In addition, we neglect the cost of downloading computation results from servers because they are relatively too small comparing with the raw data. In the following, we explain our system model in detail.

A. Data Sensing Model

The first step of data sensing for each UAV i is flying to the target position based on a certain direction and distance; this process leads to flight delay and energy consumption, denoted by $T_{t,i}^{\text{fly}}$ and $E_{t,i}^{\text{fly}}$ respectively. We denote the speed of the UAV i as v_i (unit/sec), then the flight delay $T_{t,i}^{\text{fly}}$ is

$$T_{t,i}^{\text{fly}} = \frac{d_t^i}{v_i}. \tag{1}$$

The corresponding energy consumption during the flight is represented by

$$E_{t,i}^{\text{fly}} = p_i^f \frac{d_t^i}{v_i}, \tag{2}$$

where p_i^f denotes the flight power. After arriving at the target position, UAV begins to collect data within its sensing range. We represent the sensing delay by

$$T_{t,i}^{\text{sen}} = \sum_{j=1}^{M} \beta_{i,j} \frac{b_j}{f_i^s}, \tag{3}$$

where $\beta_{i,j} \in \{0, 1\}$ indicates whether task j is within the sensing range of UAV i. If it is, $\beta_{i,j} = 1$; otherwise $\beta_{i,j} = 0$. The term f_i^s represents the sensing rate of UAV i. Without loss of generality, UAV needs to keep hovering during data sensing. The hovering power is denoted by p_i^h and the energy cost per unit for data sensing is p_i^s. Thus, the energy consumption on data sensing is

$$E_{t,i}^{\text{sen}} = p_i^h \sum_{j=1}^{M} \beta_{i,j} \frac{b_j}{f_i^s} + p_i^s \sum_{j=1}^{M} \beta_{i,j} b_j. \tag{4}$$

B. Local Computation Model

After completing the data sensing, UAV puts tasks into a local queue preparing to perform computation locally. We assume that each UAV's onboard computing facility is equipped with a single-core CPU. In other words, UAV can only execute one task at a time and the local computation time can be computed as

$$T_{t,i}^{\text{loc}} = \sum_{j=0}^{M} \beta_{i,j} \frac{(1 - \mu_t^i) l_j}{f_i^l}, \tag{5}$$

where f_i^l denotes the local CPU frequency of UAV i and l_j represents the required CPU cycles to finish task j. The term μ_t^i represents the proportion of the computation tasks offloaded to the servers, where $\mu_t^i \in [0, 1]$ and $(1 - \mu_t^i)$ is the proportion of local computation tasks.

We denote the local computation power for UAV i by p_i^l. The energy consumption on local computation denoted by $E_{t,i}^{\text{loc}}$ is represented by

$$E_{t,i}^{\text{loc}} = p_i^l \sum_{j=0}^{M} \beta_{i,j} \frac{(1 - \mu_t^i) l_j}{f_i^l}. \tag{6}$$

C. Offloading Computation Model

Considering the limited computational power of UAVs, part of the computation tasks can be offloaded to the servers on the purpose of achieving better performance. However, UAV needs to upload the task data before computation offloading. In this model, the number of servers is generally smaller than the number of UAVs so that multiple UAVs can access the same server within the same time slot and share network resources with the server.

As such, we consider the multi-device wireless interference model, similar to [7], [42], when computing the uplink transmission rate ρ_t^i of each UAV. Therefore, ρ_t^i can be expressed

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on March 29,2024 at 06:52:56 UTC from IEEE Xplore. Restrictions apply.

as follows,

$$\rho_t^i = W_{z_t^i} \log_2 \left(1 + \frac{p_i^u h_t^{i,z_t^i}}{\varpi_{z_t^i} + \sum_{j \in \mathcal{N} \setminus \{i\}: z_t^j = z_t^i} p_i^u h_t^{i,z_t^i}} \right), \quad (7)$$

where z_t^i denotes the serial number of a selected server by UAV i at time slot t, $W_{z_t^i}$ is the channel bandwidth, $\varpi_{z_t^i}$ denotes noise power, and p_i^u is the transmission power of uplink data transmissions of UAV i. The term $h_t^{i,z_t^i} = d_{i,z_t^i}^{-pl}$ is the channel gain decided by the path-loss exponent pl and the distance $d_{i,z_t^i}^{-pl}$ between z_t^i and i. And $j \in \mathcal{N} \setminus \{i\} : z_t^j = z_t^i$ indicates the set of other UAVs accessed to the same server z_t^i with i. Then, the transmission time for UAV i can be written as

$$T_{t,i}^{\text{tran}} = \sum_{j=0}^{M} \beta_{i,j} \frac{\mu_t^i b_j}{\rho_t^i}. \quad (8)$$

Accordingly, the energy consumption of data transmission is given by

$$E_{t,i}^{\text{tran}} = p_i^t T_i^{tran} + L_i. \quad (9)$$

Similar to the local computation model, we use $f_{z_t^i}^o$ and $p_{z_t^i}^o$ to denote the CPU frequency and the power of server z_t^i, respectively. Therefore, the offloading computation time $T_{t,i}^{\text{off}}$ can be calculated as

$$T_{t,i}^{\text{off}} = \sum_{j=0}^{M} \beta_{i,j} \frac{\mu_t^i l_j}{f_{z_t^i}^o}, \quad (10)$$

and the energy consumption $E_{t,i}^{\text{off}}$ can be calculated as

$$E_{t,i}^{\text{off}} = p_{z_t^i}^o \sum_{j=0}^{M} \beta_{i,j} \frac{\mu_t^i l_j}{f_{z_t^i}^o}. \quad (11)$$

IV. Problem Formulation

In this section, we formulate the multi-UAV sensing and computation problem as a partially-observable Markov decision process (POMDP) which can be defined as a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \Omega, \mathcal{O}, \gamma)$, where $\mathcal{T}$ and Ω are the state transition function and the observation function, respectively. Both $\mathcal{T}$ and Ω are predefined in the environment. The term γ is a discount factor of the future reward. In POMDP, each agent obtains an observation $o_t \in \mathcal{O}$ based on the current system state $s_t \in \mathcal{S}$ and then selects an action $a_t \in \mathcal{A}$ according to their policy $\pi_i(a_t|o_t)$. As the consequence of the action, the agent will get the reward and next observation from the environment. The goal of agents is to maximize the expected return $\mathbb{E}_{\pi_i}[\sum_{t=1}^{\infty} \gamma^{t-1} r^t]$. More explicitly, the state, observation, action, and reward are defined as follows.

A. State Space

The system state space is represented by $\mathcal{S} = \{S^{\text{task}}, S^{\text{ser}}, S^{\text{uav}}\}$, which consists of three subsets described as follows:

1) S^{task} is the collection of time-varying task information. The state of each task $j (j \in \mathcal{M})$ is denoted by a tuple $(x_j^{\text{task}}, y_j^{\text{task}}, b_j, l_j)$, where the first two elements indicate the coordinates of task j, b_j denotes the data amount, and l_j is the required CPU cycles to finish the task computation.
2) $S^{\text{ser}} = \{(W_k, f_k^o)|_{k \in \mathcal{K}}\}$ is the collection of server information, where W_k is the bandwidth changed with time-varying channels and f_k^o denotes the CPU frequency of the server.
3) S^{uav} is the state of all UAVs. Set S^{uav} is composed of UAVs' coordinates, i.e., $S^{\text{uav}} = \{(x_i^{\text{uav}}, y_i^{\text{uav}})|_{i \in \mathcal{N}}\}$.

B. Observation Space

The agent (i.e., the UAV) has a limited observation range, that is, UAV can only receive the state message from other UAVs within its broadcasting range. Thus, the observation space of the agent is usually a subset of the state space. Herein, we denote the observation space by $\mathcal{O} = \{\mathbb{T}_i, \mathbb{U}_i, S^{\text{ser}} | i \in \mathcal{N}\}$, where $\mathbb{T}_i \subset S^{\text{task}}$ is the collection of the state of closest tasks $M' (M' \leq M)$ within the observation range. Similarly, $\mathbb{U}_i \subset S^{\text{uav}}$ represents the collection of the state of closest UAVs $N' (N' \leq N)$. In our model, the collection of server information S^{ser} can be obtained by all UAVs.

C. Action Space

According to the policy and the observed system state, each UAV makes a joint action including both data sensing and computation offloading at each time slot. Since both sensing and computation contribute to task costs, it is reasonable to jointly consider them together. Specifically, we define the action space as $\mathcal{A} = \{(d_i, \theta_i, \mu_i, z_i)|_{i \in \mathcal{N}}\}$, where d_i is flight distance and θ_i is flight direction. Both of them decide the position of UAV i at the end of each time slot. As defined in Section III, the last two elements, μ_i and z_i denote the proportion of offloading tasks and the serial number of selected servers, respectively. Both μ_i and z_i constitute the computation strategy of UAV i.

D. Reward Function

For each state, the agents choose their actions and then the environment returns a reward which is a function taking the state-action pair as input, i.e., $\mathcal{S} \times \mathcal{A} \rightarrow \mathcal{R}$. Our objective is to reduce the cost of UAVs while completing as many tasks as possible. Accordingly, the system reward $\mathcal{R}_t = \{r_t^i | i \in \mathcal{N}\}$ can be expressed as

$$r_t^i = G_t^i - C_t^i - P_t^i, \quad (12)$$

where G_t^i represents the gain for UAV i obtained by completing tasks in epoch t, C_t^i denotes the total cost of time and energy consumption for UAV i, and P_t^i is a penalty. According to the equations defined in Section III, C_t^i can be written

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on March 29,2024 at 06:52:56 UTC from IEEE Xplore. Restrictions apply.

as the total sum of time and energy consumption, i.e.,

$$C_t^i = \kappa^T (T_{t,i}^{\text{fly}} + T_{t,i}^{\text{sen}} + T_{t,i}^{\text{loc}} + T_{t,i}^{\text{tran}} + T_{t,i}^{\text{off}}) + \kappa^E (E_{t,i}^{\text{fly}} + E_{t,i}^{\text{sen}} + E_{t,i}^{\text{loc}} + E_{t,i}^{\text{tran}} + E_{t,i}^{\text{off}}), \tag{13}$$

where κ^T and κ^E indicate the weights prioritizing the time or energy consumption and the sum of them is equal to 1, i.e. $\kappa^T + \kappa^E = 1$. The term G_t^i indicates the gain for UAV i obtained by completing tasks within epoch t. Without loss of generality, we assume that G_t^i correlating with the amount of sensing data is firstly completed by UAV i. Note that if a UAV finishes the task later than others, it will get zero gain. Therefore, G_t^i can be calculated by

$$G_t^i = \sum_{j=0}^{M} \zeta_{i,j} g(b_j, l_j), \tag{14}$$

where $\zeta_{i,j} \in \{0, 1\}$ denotes whether task j is first finished by UAV i. If it is, $\zeta_{i,j} = 1$ otherwise $\zeta_{i,j} = 0$. In Eq. (14), $g(b_j, l_j)$ is a gain function associated with the amount of sensing data b_j and computation power l_j required by task j. To guide the training of agents, we also set a penalty P_t^i that considers two cases: 1) flying out the defined region, and 2) existing uncompleted tasks. Therefore, we define P_t^i as

$$P_t^i = \delta_t^i P^{\text{fly}} + n_t^u P^{\text{unf}}, \tag{15}$$

where $\delta_t^i = 1$ indicates that UAV i flies out of the given region, and otherwise $\delta_t^i = 0$. The term P^{fly} is the penalty value of the UAV flying out of the region and n_t^u represents the number of tasks that are not executed in epoch t. The term P^{unf} denotes a penalty factor for each unfinished task.

V. Algorithm Design

To address the above problem, we propose an improved DRL algorithm based on the actor-critic framework. In detail, we regard each UAV as an agent and apply the multi-agent DRL algorithm to reduce the control-response delay to UAVs, thereby improving the system performance. Since communication among multiple agents may cause high delay and energy cost [43], instead of adopting the explicit channel for information exchange, we leverage the critic network to model other agents for seeking a balanced state during the concurrently-learning process of all UAVs. In addition, we introduce the attention mechanism to enhance the convergence performance in model training phases.

A. Multi-Agent Actor-Critic Framework

The proposed multi-agent DRL algorithm is based on the actor-critic framework, as illustrated in Fig. 2. The network structure consists of two components: an actor network and a critic network. To stabilize the training process, we adopt two additional neural networks for each actor network and each critic network. In particular, there are an online actor network and a target actor network in each actor network. Similarly, there are an online critic network and a target critic network in each critic network. As the policy function of agents, the online actor network outputs specific actions to control agents. Like most of the state-of-the-art multi-agent DRL solutions [43], [38], we train a separate actor network for each UAV. Based on observations and actions from all agents, the critic network outputs state-action values for each agent, which can be used to evaluate policy performance in training phase. Detailed interactions and network training methods are presented as follows.

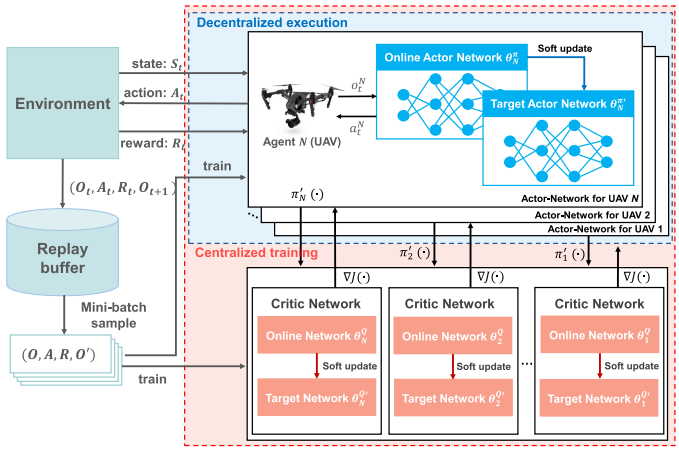


Fig. 2. The actor-critic framework.

At a time slot t, each UAV obtains observation o_t^i based on the current system state S_t and its observation range r_s^i, and then inputs the observation into the online actor network. As the feedback, the actor network outputs action a_t^i to be executed at UAV i in the current time slot. To explore the optimal strategy and avoid the local optimum, we add a noise derived from the Ornstein-Uhlenbeck (OU) process to action before execution. Then, the environment generates the state of the next time slot S_{t+1} according to the action of all agents and the predefined state transition function Ω, and returns an immediate reward $R_t = \{r_t^1, \ldots, r_t^N\}$ to each agent. Once an iteration is completed, the generated experience tuples (O_t, A_t, R_t, O_{t+1}) will be stored in the replay buffer for parameters update. Note that newly generated tuples will replace the oldest ones when the reply buffer is full due to its limited capacity.

In a single-agent environment, the critic network maps the system state and action pair (S_t, A_t) to the long-term expected reward. However, in a multi-agent environment, agents can only obtain their local observations. Considering the energy cost and delay in communication, we do not adopt the explicit channel for information exchange like [44]. Instead, we use the critic network to model other agents in the training process, thereby evaluating the performance of the actor-network. Therefore, the input of the critic network contains the value of observations and actions from all agents at one time slot, i.e., $Q^\pi(O_t, a_t^1, \ldots, a_t^N)$; while the network parameters are updated by using the interactive experience of all agents in a centralized training mode. The specific updating process is as follows.

We update the parameters of both actor network and critic network within a period of ψ. More specifically, after each ψ

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on March 29,2024 at 06:52:56 UTC from IEEE Xplore. Restrictions apply.

iteration, we randomly select a batch of samples from the replay buffer to update the network parameters. Given the sampled experience data $(O_k, A_k, R_k, O_{k+1}), k = 1, \ldots, H$, the online critic network updates its parameters θ_i^Q by minimizing the loss function given by

$$L(\theta_i^Q) = \frac{1}{H}\sum_{k=1}^{H}\left(y_k^i - Q_i^\pi\left(O_k, a_k^1, \ldots, a_k^N | \theta_i^Q\right)\right)^2, \quad (16)$$

where the target value y_k^i is computed by

$$y_k^i = r_k^i + \gamma Q_i^\pi\left(O_{k+1}, a_{k+1}^1, \ldots, a_{k+1}^N | \theta_i^{Q'}\right). \quad (17)$$

The action $a_{k+1}^i = \pi(o_{k+1}^i | \theta_i^{\pi'})$ is the output of the target actor network for the next time slot. In our framework, the critic network is trained with extra information from the policies of all agents, while the parameters of the actor network are only simply updated with local information. Once the training process is completed, each local actor will be used at the execution phase and act in a decentralized manner. For the online actor network, we update its parameters θ_i^π using policy gradients as

$$\nabla_{\theta_i^\pi} J \approx \frac{1}{H}\sum_{k=1}^{H} \nabla_{\theta_i^\pi} \pi\left(o_k^i | \theta_i^\pi\right) \nabla_{a_k^i} Q_i^\pi\left(O_k, a_k^1, \ldots, a_k^N \mid \theta_i^Q\right). \quad (18)$$

Finally, we adopt the "soft update" method to update the target network that approximates the parameter of online network with step $\tau << 1$:

$$\begin{aligned} \theta_i^{Q'} &\leftarrow \tau\theta_i^Q + (1-\tau)\theta_i^{Q'}, \\ \theta_i^{\pi'} &\leftarrow \tau\theta_i^\pi + (1-\tau)\theta_i^{\pi'}. \end{aligned} \quad (19)$$

B. Attention Critic Network

As mentioned above, we train the actor network for each UAV, which takes local observation as the input and outputs cooperative actions by modeling other agents' strategy with the critic network. At present, agent-modeling algorithms like MADDPG [43] use the critic network composed of a full connection layer to model other agents, which can well behave in an environment with fewer agents. However, with the increasing number of agents in the system, it is difficult for the critic network with a simple fully-connected layer to deal with the input with ever-increasing dimensions, thereby leading to the slow or even difficult convergence of the critic network and consequently affecting the training speed of the actor network.

To address this issue, we introduce the attention mechanism into the critic network, which helps agents pay different attention to other agents when modeling with the critic network. The attention mechanism is widely applied in natural language processing, computer vision, and other domains. In recent years, DRL researchers adopt the attention mechanism to improve the DRL algorithms, e.g., [20] and [21]. Long *et al.* [20] introduced the attention mechanism to scale up multi-agent DRL algorithms by supporting the increase in the number of training agents for a dynamic environment. Iqbal *et al.* [21] combined the attention mechanism with soft actor-critic algorithms to apply in cooperative multi-agent settings. And the attention mechanism was demonstrated to have important, positive effects on both the system reward and the learning speed for complex multi-agent environments. The intuition behind our idea comes from the fact that the nearby agents have greater influence and should be paid more attention. More explicitly, UAVs can obtain a similar observation to the nearby UAVs and execute sensing tasks in the same area, which affects the revenue of their tasks. In other words, the action from the closer UAVs can have a greater impact on the reward.

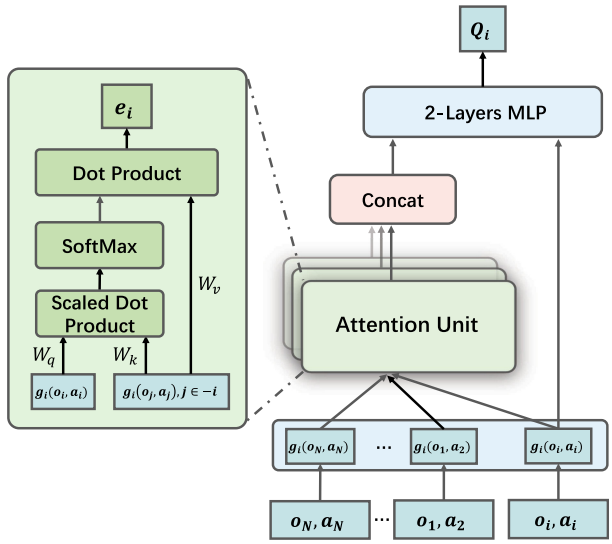


Fig. 3. The attention-critic network architecture.

Motivated by this intuition, we propose to embed multi-attention units to the critic network. Although all the above-mentioned studies adopt a critic network to evaluate the agents and thereby reducing the training time due to less network updates, leveraging such a critic network cannot provide accurate performance evaluation for UAVs with different properties in our framework. Therefore, as shown in Fig. 3, we train one critic network embedded with a multi-attention unit for each UAV. In Fig. 3, every critic network has a single input layer, two hidden layers, and a single output layer. We place the attention unit in between the input layer and two-hidden layer networks. Note that the first hidden layer receives the input of UAV i's feature value ϑ_i, and also gets the attention values of other UAVs (see (23)).

When calculating the Q-value for UAV i, we firstly input the observation-action pair (o_j, a_j) of all agents into a one-layer multi-layer perceptron (MLP) and generate the extracted feature value as

$$\vartheta_j = g_i(o_j, a_j), \quad (20)$$

where g_i is the one-layer MLP function. Subsequently, the attention unit calculates the attention value with these

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on March 29,2024 at 06:52:56 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1. Proposed algorithm: AT-MARL

1 **Initialize:** actor network learning rate α_a, critic network learning rate α_c, soft-update parameter τ, discount factor γ, replay buffer B, update interval ψ, batch size H;
2 **for** UAV $i \in \mathcal{N}$ **do**
3 Initialize UAV i's actor network and critic network with parameters θ_i^{π} and θ_i^{Q};
4 Initialize target actor network and target critic network with parameters $\theta_i^{\pi'} := \theta_i^{\pi}$ and $\theta_i^{Q'} := \theta_i^{Q}$;
5 **end**
6 **for** episode $= 1, 2, \ldots, E_{\max}$ **do**
7 Initialize system environment and obtain state S_0 and observation $O_0 = (o_0^1, o_0^2, \ldots, o_0^N)$;
8 **for** epoch $t = 0, 1, \ldots, T_{\max} - 1$ **do**
9 **for** UAV $i \in \mathcal{N}$ **do**
10 Generate an action $a_t^i = \pi^i(o_t^i|\theta_i^{\pi})$ from actor network and add OU noise for exploration;
11 **end**
12 Execute all actions $A_t = (a_t^1, a_t^2, \ldots, a_t^N)$;
13 Obtain the next state S_{t+1}, next observation $O_{t+1} = (o_{t+1}^1, o_{t+1}^2, \ldots, o_{t+1}^N)$ and current reward $R_t = (r_t^1, r_t^2, \ldots, r_t^N)$ from the environment;
14 Add transition (O_t, A_t, R_t, O_{t+1}) into buffer B;
15 $S_t = S_{t+1}, O_t = O_{t+1}$;
16 **if** t mod $\psi = 0$ **then**
17 Sample a mini-batch transition from buffer B;
18 **for** UAV $i \in \mathcal{N}$ **do**
19 Update the parameter of critic network by minimizing loss computed with (16);
20 Update the parameter of actor network using gradient calculated with (18);
21 Soft-update the parameters of target network using (19);
22 **end**
23 **end**
24 **end**
25 **end**

feature values. In this paper, the feature value of UAV i is *query* and of others are *key* and *value*. The detailed calculating process can be divided into three steps as described as follows.

The first step is similarity calculation. In this work, we use the dot product to calculate the similarity between different UAVs. Compared with MLP and other methods, the dot product can improve the operation speed and achieve good results with highly-optimized codes [45]. Note that *query* is UAV i's feature value ϑ_i while *key* is another UAV $j(j \neq i)$'s feature value. Accordingly, the similarity between UAV i and UAV j is calculated as

$$\mathrm{sim}_j = \vartheta_j^T W_k^T W_q \vartheta_i, \tag{21}$$

where W_k and W_q are the scaled matrixes for the key and the query, respectively, which can be used to prevent vanishing gradients [21].

Next, we adopt softmax function to normalize the similarity as calculated above, and then generate the attention weight α_j of UAV $j(j \in -i)$

$$\alpha_j = \mathrm{softmax}\big(\mathrm{sim}_j\big) = \frac{\exp\big(\mathrm{sim}_j\big)}{\sum_{k \neq j} \exp(\mathrm{sim}_k)}. \tag{22}$$

Finally, we use attention weight α_j to sum the *value*s. The *value* is the same as the *key*, which is the extracted feature of other UAVs ϑ_j. Therefore, the attention value of UAV i can be expressed as

$$e_i = \sum_{j \neq i} \alpha_j W_v \vartheta_j, \tag{23}$$

where W_v is the scaled matrix for *value*.

In addition, we introduce the multi-head attention. In particular, we use K attention units to perform the attention function in parallel and concatenate the calculated attention value before inputting into the two-layer MLP of the critic network. The final output of the critic network, called Q-value, can be expressed as

$$Q_i(O, a_1, \ldots, a_n) = f_i(\vartheta_i, \mathrm{concat}(e_i^1, \ldots, e_i^k)), \tag{24}$$

where the concat function concatenates all the vector of attention values obtained from multiple attention units into a single vector and f_i is the two-layers MLP that takes the concatenated attention value and its feature as input, and outputs the final Q-value, as shown in Fig. 3.

C. Algorithm Description

We refer to the proposed algorithm for UAV-assisted MCS systems as AT-MARL. For most DRL algorithms, the critic network is a fully-connected neural network. However, applying the critic network with a fully-connected structure in the multi-agent environment can be hard to obtain accurate approximate values. Therefore, we improve our AT-MARL algorithm by embedding the attention units to each critic network. Thus, each UAV pays different attentions to other UAVs so that the actor network can train better with more accurate values. The details of AT-MARL are explained in Algorithm V-B. At first, we define the related hyper-parameters, including the network learning rate α_a and α_c, update interval ψ, and so on (Line 1). For each UAV, we initialize four DNNs, i.e., actor network, critic network, and the two corresponding target networks (Line 2-5). Note that the parameter of embedded attention units is also initialized in the critic network. In the following, we describe how to train these DNNs in the proposed environment.

At the beginning of each episode, we randomly initialize the UAV-assisted MCS system with state S_0 and each UAV obtains its observation o_0^i based on S_0 (Line 7). Without loss of generality, we divide an episode into $T_{\max}$ epochs. For each epoch t, UAVs firstly get an action based on the current observation and their policy. Before execution, a noise generated from the OU process is added into the action for exploration (Line 9-11). Afterward, all UAVs execute their joint actions including data sensing and computation offloading (Line 12). Note that if UAV i flies out of the map after executing the generated action, it will abandon the flying action and stay. Next,

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on March 29,2024 at 06:52:56 UTC from IEEE Xplore. Restrictions apply.

each UAV receives the immediate reward r_t^i from the environment as well as the next epoch observation o_{t+1}^i that is based on the next state S_{t+1} (Line 13). Finally, the experience tuple (O_t, A_t, R_t, O_{t+1}) generated from the above iteration is stored into the reply buffer for parameters update (Line 14).

In every ψ epoch, we use the stored experience to update the parameters of DNNs. Firstly, we sample a mini-batch of H transitions from the replay buffer B (Line 17). Given the sampled data, each UAV i subsequently updates the parameter of the critic network by minimizing the loss value which can be calculated with (16) (Line 19), and updates the parameter of the actor network by gradient ascent which is calculated with (18) (Line 20). In the end, the parameters of the target network are updated using (19) (Line 21).

The above iteration and network training will last for $E_{\max}$ episodes to ensure the convergence of the network. Then, we only retain the online actor network that is used to generate the action for UAVs. In the testing process, each UAV inputs the observation into the trained actor network and executes the output action.

VI. Performance Evaluation

In this section, we evaluate the performance of AT-MARL algorithm through simulations and also conduct an experiment in real-world scenarios. We first present the experiment setup including the training environment and DNN hyper-parameters. Then, we analyze the convergence performance and moving trajectory in simulations. Furthermore, we compare the proposed algorithm with four baseline algorithms under various environmental settings. Finally, we develop a real-world UAV test platform to evaluate the performance of our algorithm.

A. Experiment Setup

For model training, we use PyTorch 1.7.0 to implement the AT-MARL algorithm and train each agent's DNN (i.e., the actor network and critic network) with four NVIDIA GTX 1080Ti graphic cards. We assume that each agent has the same DNN architecture. More specifically, both actor network and critic network have one input layer, two hidden layers, and one output layer, where the first and second hidden layers have 128 and 256 neurons, respectively. We use Adam optimizer to optimize the parameter of DNN with a learning rate 0.001 and update the DNN once per 25 epochs. The training batch size and replay buffer size are set to 512 and 20,000, respectively. The discount factor γ for the future reward is 0.99 and the soft-update rate for the target network τ is set to 0.05. Table I shows the parameters of neural networks in the training settings.

In simulations, we keep the task arrival rates constant at 20 per time slot. These tasks are uniformly distributed in the target area, which is a 2-dimensional square with a size of 100×100 units. The associated data amount of the tasks follows a uniform distribution between 200 MB and 400 MB. Besides, we set three edge servers and one cloud server in the target position to provide computation offloading services, where the values of the computing power p^o are chosen for the edge server with 200 W and the cloud server with 500 W, respectively. The CPU frequency f^o of edge servers ranges from $[10,\ 20]$ GHz, and the uplink bandwidth ranges from $[6,\ 8]$ MHz, while the corresponding values of the cloud server are set to 40 GHz and 1 MHz, respectively. For the computation tasks, the cost of CPU cycles is assumed to range from $[300,\ 600]$ Gigacycles. In our experiment, 5 UAVs equipped with 50 W power onboard computers participate in the crowdsensing. For data sensing, the flight speed of UAVs ranges from $[1,\ 2.5]$ unit/s, each of which has a coverage sensing range of 20 units, and the energy cost per unit is 0.9 joules. The data sensing rate of each UAV is 1.5 MB/s and the CPU frequency ranges from $[2,\ 3]$ GHz. For data transmission, the channel bandwidth W_{z_t} is 20 MHz, the transmit power p^u is 0.1 W, the background noise ϖ_{z_t} is set to -100 dBm, and the path-loss factor pl is 4.

TABLE I
The Parameters of Neural Networks for Training

Parameter	Value
The learning rate of actor networks	0.001
The learning rate of critic networks	0.002
The soft update coefficient of target networks	0.01
Replay buffer size	20,000
Number of sampled experience sampling	128
Random seed	1024
Number of heads in multi-head attention	2

The baseline algorithms for comparison with our proposed AT-MARL are listed as follows:

1) *Randomized algorithm:* At each time slot, the randomized algorithm randomly generates actions for all UAVs based on the set range of action values. Here, we set the random seed to 1024.

2) *Genetic algorithm:* For action making, the genetic algorithm needs to encode the action of UAVs and randomly generates multiple segments of different action genes. These genes are then iterated for a certain number of reproductions including crossover, mutation, and selection. Finally, in a time slot, the action of UAVs is the highest adaptive gene selected by the fitness function.

Specifically, we use 8-bit binary numbers to represent the value of an individual element in actions for UAV i. As such, each individual DNA length is $4 \times 8 = 32$, for example, we can set $[00000001, 00000000, 00000010, 00000011]$ to be flight distance $d_{\max} \times \frac{1}{1024}$, flight angle $2\pi \times \frac{0}{1024}$, transfer scale $1 \times \frac{2}{1024}$, and accessible server number $|K| \times \frac{3}{1024}$. Moreover, we choose 100 population and 100 iterations. We set fitness function as the reward function, and take individual with the highest fitness in the final generation as UAV i's action value.

3) *DDPG algorithm:* DDPG is a single-agent DRL algorithm regarding the whole system as an independent agent, in which the centralized actor network is used as a UAV controller that inputs system state and outputs action for all UAVs at each time slot. The neural network in experiments has one input layer, three hidden layers, and one output layer. Herein,

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on March 29,2024 at 06:52:56 UTC from IEEE Xplore. Restrictions apply.

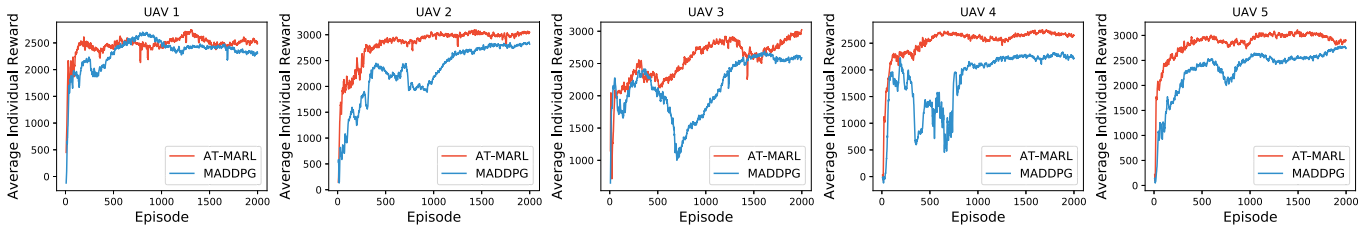


Fig. 4. Evaluation of training performance on each agent.

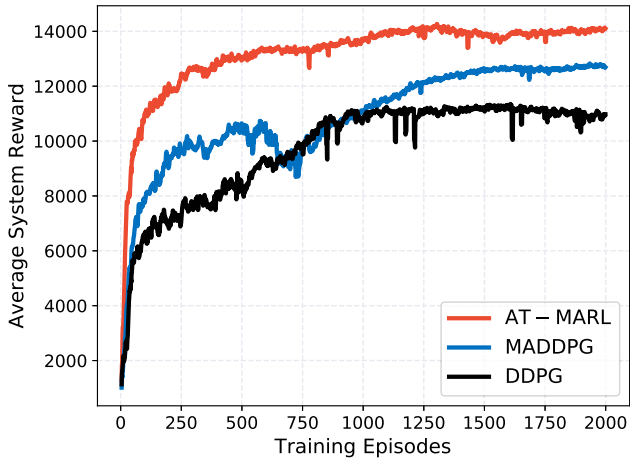


Fig. 5. Evaluation of convergence performance.

the three hidden layers have 128, 256, and 128 neurons, respectively.

4) *MADDPG algorithm:* Similar to AT-MARL, MADDPG is a multi-agent DRL algorithm making decisions based on the actor network deployed in each UAV. In each time slot, every UAV inputs the observation from the current system state into the online actor network, and in return gets a determinate action. In experiments, we set the parameters of neural networks for MADDPG the same as the AT-MARL algorithm.

B. Convergence Performance

We first evaluate the convergence performance of the AT-MARL algorithm. Specifically, we first evaluate the individual reward of each agent during training process. We then evaluate the system reward versus task completion rate of the AT-MARL algorithm with comparison with two state-of-the-art baselines, i.e., DDPG [46] and MADDPG [43].

In this set of experiments, we consider a simulation environment with 5 UAVs and 20 tasks for each time slot. Fig. 4 shows the training performance of five individual UAVs. It can be seen that the average individual reward of each agent increases obviously during the training process and then reaches to a stable value (i.e., convergence) while each agent has a different convergence value from each other. This is because each UAV varies in its individual property, which causes a different strategic performance and thus obtaining a different reward. Compared with MADDPG, each agent in the AT-MARL algorithm shows a better training performance in terms of a higher individual reward and a fast convergence speed. In terms of the stability during the training process, MADDPG has jitter problems in the last four agents occurring between 500 and 1,000 training episodes. This is because the low strategic performance of a single agent may lead to an overlapped sensing range for multiple UAVs, thereby resulting in unnecessary costs by performing sensing tasks in the same area and consequently obtaining lower rewards. Fortunately, the incorporated attention mechanism in the AT-MARL algorithm reduces the possibility of overlapped sensing range with adjacent UAVs to avoid costing on repeated tasks; thus, AT-MARL keeps a higher system reward and a better stability than MADDPG.

Fig. 5 shows that the average system reward of DDPG, MADDPG, and AT-MARL. We observe that DDPG, MADDPG, and AT-MARL all achieve a drastically-increased the average system reward during the early stage of training and then quickly reach the convergence. It is obvious that both two multi-agent algorithms MADDPG and AT-MARL have a better convergence performance than the single-agent DDPG algorithm. For a further comparison with MADDPG, the proposed AT-MARL algorithm shows more robust and yields a better performance in terms of average system reward and training episodes. This is because the AT-MARL algorithm leverages the attention mechanism to allow the actor network to quickly obtain a more accurate value of performance evaluations during training, thus improving the training performance of the actor network.

C. Performance Evaluation Under Various Environment Settings

In this section, we evaluate the AT-MARL algorithm with comparison with four baselines in terms of multiple performance metrics. In detail, we conduct simulations under four groups of experimental settings with 3, 5, 8, and 10 UAVs, respectively. The sensing task cycle of UAVs is set to 10, each of which includes 1,000 time slots.

Fig. 6 plots the average system utility of our AT-MARL with other four baselines. We observe from Fig. 6 that our AT-MARL has nearly the same average system utility as other three baselines, i.e., DDPG, MADDPG, Genetic algorithms when the number of UAVs is 3. However, our AT-MARL soon outperforms other baselines with the increased number of UAVs. The reason can be explained as follows: 1) it is difficult for Randomized algorithm and Genetic algorithm to obtain an optimal solution since the solution space of the optimization of them increases exponentially with the increased

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on March 29,2024 at 06:52:56 UTC from IEEE Xplore. Restrictions apply.

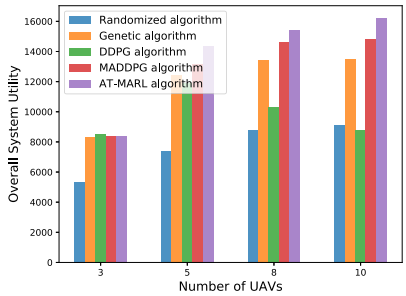


Fig. 6. Overall system utility of AT-MARL with other baselines under different numbers of UAVs.

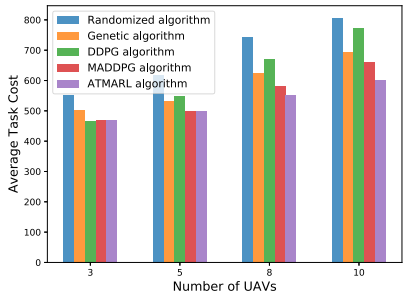


Fig. 8. Average task cost of AT-MARL with other baselines under different numbers of UAVs.

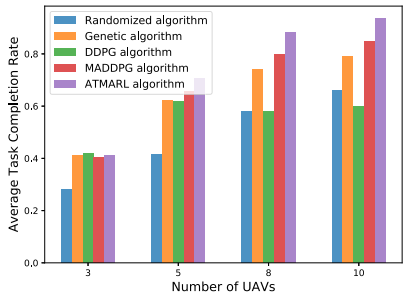


Fig. 7. Task completion rate of AT-MARL with other baselines under different numbers of UAVs.

number of UAVs; 2) the increased number of UAVs enlarges the input and output dimensions of the actor network so that the single-agent DDPG algorithm can be difficult to learn optimal strategies or even cannot converge in training; 3) the critic network in the MADDPG algorithm becomes harder to obtain essential information from a complex environment caused by the increased number of UAVs. Thanks to the attention mechanism, our AT-MARL can pay more attention to those nearby UAVs, which have greater influences on the system state. Thus, our AT-MARL can get an accurate Q-value in the environment with the increased UAVs, thereby achieving the best performance.

We then evaluate the task completion rate of our algorithm, as illustrated in Fig. 7. The results show a similar trend to the average system utility as that of in Fig. 6. This demonstrates that our algorithm can obtain a high system reward, which is not achieved by reducing costs in a manner of avoiding sensing tasks. What's more, compared with baselines, our algorithm displays roughly the same results as in Fig. 6 in terms of task completion rate. The reason can refer to the above-mentioned discussions. In summary, we find that AT-MARL outperforms other benchmark algorithms in performance closely related to the system objectives, and still maintains high performance and has better scalability when the number of UAVs increases.

Finally, we evaluate the average cost of each task with different numbers of UAVs. As shown in Fig. 8, the AT-MARL algorithm demonstrates the same average task cost of each task as DDPG and MADDPG under the settings of 3 UAVs and 5 UAVs. However, when the number of UAVs increases, the AT-MARL algorithm outperforms other baselines. The reason is because only the first UAV completing the task can receive the task revenue whereas one task is more likely to be repeatedly performed by multiple UAVs when there are more UAVs in the system. In contrast, the incorporated attention mechanism in AT-MARL helps to reduce the possibility of overlapped sensing tasks with adjacent UAVs, thereby avoiding costs on repeated tasks. Consequently, AT-MARL demonstrates a relatively lower growth in terms of the average task cost.

D. Moving Trajectory Analysis

In the following, we simulate the flight process to analyze the flight trajectory of the UAVs while performing MCS tasks.

The simulation results are shown in Fig. 9, where the number of UAVs and tasks are 5 and 20, respectively, and the flight trajectory is based on 4 randomly-selected time slots. In Fig. 9, star points and circular points denote the locations of UAVs and the location of sensing tasks, respectively, the colored circles represents the data-collection regions. We observe from Fig. 9 that at the beginning of a MCS task, as illustrated in the grey circles, multiple sensing tasks are out of the sensing scope of UAVs due to their limited sensing range. Meanwhile, UAVs obtain action values of the current time slot. After acting on tasks, the new state such as the UAV locations and their coverage regions are depicted as the blue circles. The results show that all UAVs are closer to task-intensive areas after flying at a certain distance and the number of tasks within the sensing range increases significantly. Therefore, we can conclude that our algorithm can achieve a high system reward by controlling UAVs to complete more sensing tasks.

Fig. 10 depicts all trajectory points of UAVs in the full cycle of MCS tasks under different algorithms. The results show that our AT-MARL algorithm enables each UAV to move within a fixed region. This is because the learning strategy in AT-MARL reduces the flight costs by preventing UAVs from flying a long-distance flight during a time slot, consequently optimizing the total system rewards. Furthermore, the sensing area of each UAV in AT-MARL shows a lower overlapped rate than other three algorithms. This is because AT-MARL can reduce unnecessary costs by avoiding completing the same tasks in the same region. In contrast, Genetic algorithm (GA) shows that its trajectory covers almost everywhere in the given units, thereby inevitably increasing flight costs and affecting the average task costs of UAVs. As a

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on March 29,2024 at 06:52:56 UTC from IEEE Xplore. Restrictions apply.

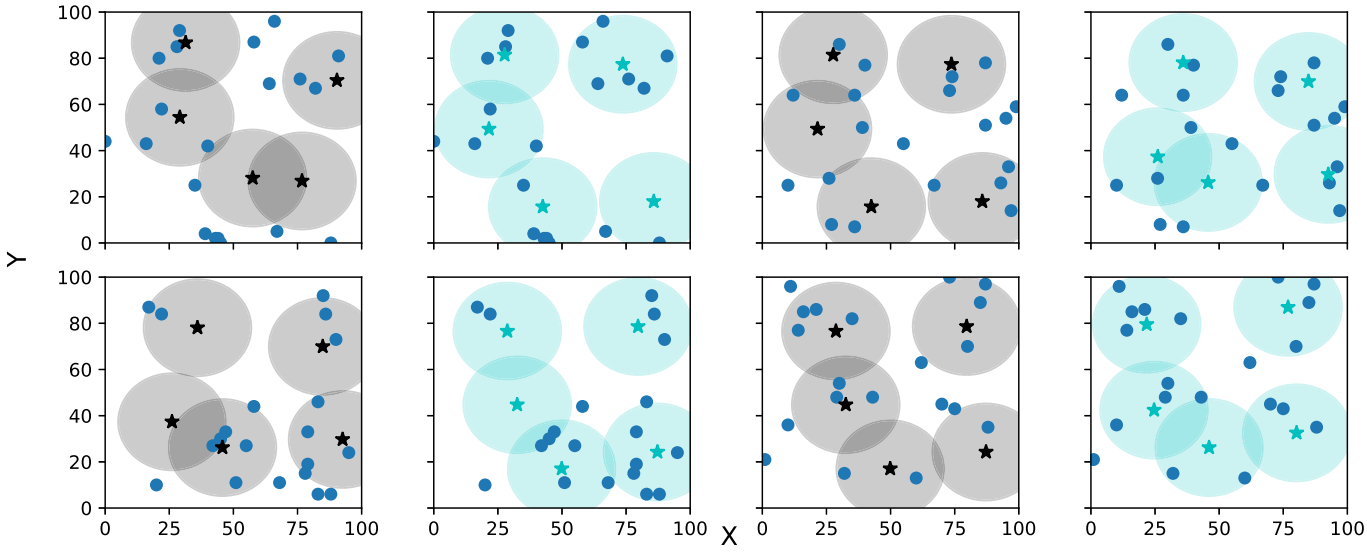


Fig. 9. The flight trajectory of UAVs and their sensing coverage transformation in 4 time slots. Representation of images from left to right, top to bottom are the location of UAVs and task points in four continuous time slots, respectively, where the images with grey sensing coverage indicate the state before taking actions while other images with blue sensing coverage represent target locations after the UAVs take output actions from the actor network.

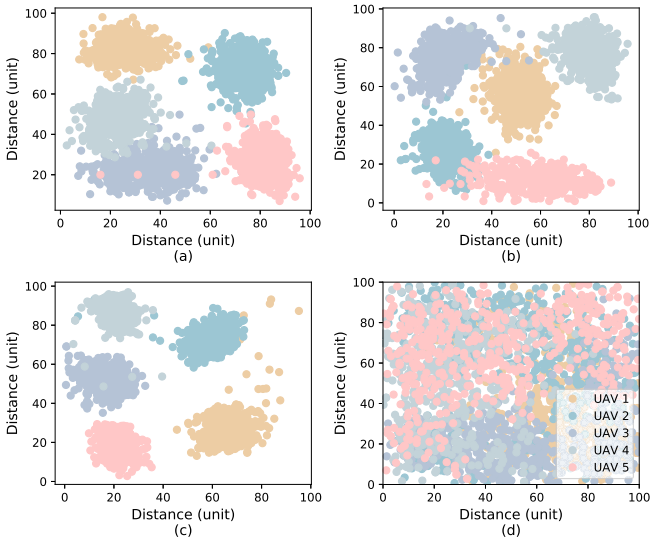


Fig. 10. Trajectories of five UAVs in one episode under different algorithms. (a) AT-MARL. (b) MADDPG. (c) DDPG. (d) GA.

result, the overall system utility is decreased. The DDPG algorithm leads to relatively small sensing areas while leaving large regions without being covered within the sensing range of UAVs so that some sensing tasks cannot be completed. Lastly, the MADDPG algorithm has fixed sensing regions for each UAV. However, there emerges a large blank area in the experimental area since some regions are too close to each other. As a result, MADDPG shows a lower task completion rate and has a smaller total system reward than our AT-MARL algorithm.

E. Experiments in Real-World Scenarios

Furthermore, to validate the availability and efficiency of the proposed AT-MARL algorithm, we also implement a real-world UAV test platform equipped with the software development kit namely onboard SDK 3.8 provided by DJI. We also conduct experiments on three actual UAVs, i.e., DJI MATRICE 600 PRO.[1] The testing results are displayed through the simulation software DJI Assistant 2 For MATRICE.

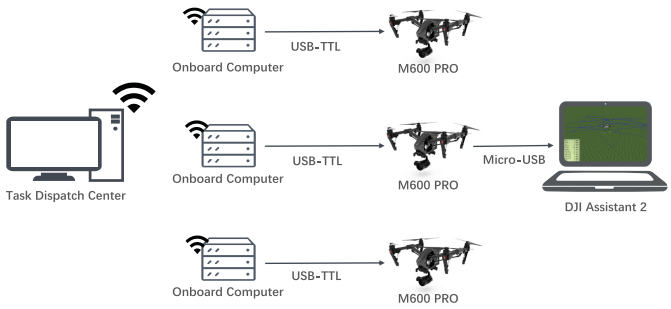
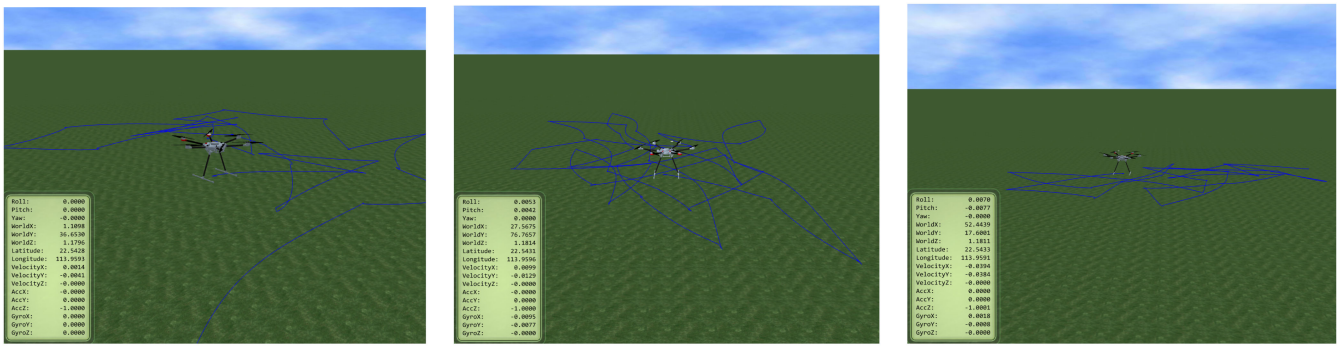


Fig. 11. The implement of the UAV test platform.

Fig. 11 depicts the implementation of the platform. In particular, a PC is used as the task dispatch center which sends the information of sensing tasks to each onboard computer through wireless connections. Note that we use Intel NUC, a small form-factor computer as the onboard computer mounted at each UAV like [47], [48]. To control the flight of UAVs, we deploy the trained actor network to the corresponding onboard computer of each UAV and connect the onboard computer and the UAV with USB-TTL. After receiving the task information from the dispatch center, the onboard computer generates the actions by the actor network and transforms them into the flight control commands for the UAV. Finally, for the convenience of the display, we also connect the UAV with a laptop through the micro-USB cable and display the flight

[1] https://www.dji.com/cn/matrice600-pro/info#specs

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on March 29,2024 at 06:52:56 UTC from IEEE Xplore. Restrictions apply.



(a) The trajectory of UAV 1 (b) The trajectory of UAV 2 (c) The trajectory of UAV 3

Fig. 12. Movement trajectory of three UAVs shown in DJI Assistant 2.

TABLE II
RUNNING TIME FOR EACH ALGORITHM

Algorithms	The number of UAVs			
	3	5	8	10
Genetic Algorithm 1	3.740	5.759	8.754	10.669
Genetic Algorithm 2	14.715	21.969	33.012	42.648
DDPG	0.252	0.261	0.269	0.262
MADDPG/AT-MARL	0.064	0.068	0.066	0.071

process of the UAV through the DJI Assistant 2 software running on the laptop.

Fig. 12 presents the experimental results. As shown in Fig. 12, we can conclude that the actor network trained by our algorithm can be deployed to the real UAVs. Meanwhile, the actor network is capable of controlling the flight of UAVs. In addition, the bottom left corner of each sub-figure is the immediate information of the UAV, such as "worldX" and "worldY," i.e., the coordinates of the UAV. Note that the coordinates of these three UAVs in Fig. 12 are (27.5675,76.7657), (1.1098,36.6530), and (52.4439,17.6001), respectively. As expected, UAVs are far away from each other. We can observe from the trajectory in Fig. 12 that each UAV moves at a certain range of its region; these results are consistent with the simulation results. Based on the above results, it is feasible and effective to deploy the proposed AT-MARL algorithm to the real-world UAVs.

In the real MCS system, the task-response delay is also an essential problem to be considered. Since the running time of our algorithm directly affects the response delay, we use the test platform to evaluate each algorithm's execution time. Table II lists the results, that is, the time required for each algorithm to generate action values for UAVs. More explicitly, for the Genetic algorithm, we compute the running time for each UAV's onboard computer to perform number of iterative calculations. For DDPG, we consider a round-trip communication delay, which includes sending the state information of UAVs to the MCS system, and then leveraging well-trained neural networks to generate and send back the action values to each UAV. For the multi-agent algorithms, like MADDPG and AT-MARL, we only calculate the execution time for obtaining action values by the neural networks deployed on the UAV itself. In particular, we test the Genetic algorithm under two settings, where Genetic Algorithm 1 chooses 100 population and 100 iterations while the corresponding values are both 200 in Genetic Algorithm 2.

From Table II, we can observe that with the increasing number of UAVs, the running time of the Genetic algorithms has also increased significantly. This is because the length of gene fragments in Genetic algorithms increases with the ever-increasing number of agents, thus increasing the calculation time of fitness evaluation. Similarly, the running time of the Genetic algorithms also increases significantly as the increase of population and iterations. In contrast, DRL algorithms generate actions of the UAVs by neural networks instead of multiple rounds of iterative calculation, thereby reducing the computational complexity. In addition, compared with the single-agent DRL algorithm (i.e., DDPG), both our algorithm and MADDPG (i.e., multi-agent DRL algorithms) have lower running time. This is because the UAV in multi-agent DRL algorithm is directly controlled by a neural network deployed on its onboard computer. The distributed control avoids the communications between UAVs and the centralized controller for exchanging UAV states, thereby resulting in a lower running time. In summary, the AT-MARL algorithm has a low running time so that it is more suitable for the real UAV-assisted MCS system.

VII. CONCLUSION

In this paper, we have studied the joint problem of data sensing and computation offloading in the UAV-assisted MCS system. With the objective of maximizing the overall system utility and catering for the dynamic environment, we formulate the joint optimization problem as a POMDP and employ the multi-agent DRL to tackle this problem. In particular, we propose a distributed multi-agent algorithm called AT-MARL by extending the single-agent actor-critic framework to the multi-agent framework. To cope with the increasing number of UAVs, AT-MARL incorporates the attention mechanism, thereby improving the performance of distributed UAV agents. Evaluation results have demonstrated the effectiveness of our AT-MARL. More specifically, we have the following observations: 1) AT-MARL quickly obtains better system rewards and yields the best convergence performance and

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on March 29,2024 at 06:52:56 UTC from IEEE Xplore. Restrictions apply.

multi-agent algorithms (including our AT-MARL) perform better performance than the single-agent DDPG algorithms; 2) the joint optimization improves the task completion rate while reducing system costs like the time and energy thereby achieving the maximum of long-term rewards.

In this article, the electrical power of UAV is assumed to be sufficient during a task period. In more general conditions, how to schedule battery charging once UAV runs out of power should be considered. Furthermore, a fixed amount of the sensing task is modeled, which should be different during each task period. Similar to most DRL algorithms, a fully-connected layer network with a fixed input dimension is adopted in actor network, which requires to relearn strategies when there are any changes with UAV numbers. Future studies include leveraging Long Short-Term Memory (LSTM) networks or embedding attention units into the actor network to deal with varying input sizes for UAV crowdsensing.

REFERENCES

[1] C. H. Liu, C. Piao, and J. Tang, "Energy-efficient UAV crowdsensing with multiple charging stations by deep learning," in *Proc. IEEE INFOCOM IEEE Conf. Comput. Commun.*, 2020, pp. 199–208.

[2] A. Capponi, C. Fiandrino, B. Kantarci, L. Foschini, D. Kliazovich, and P. Bouvry, "A survey on mobile crowdsensing systems: Challenges, solutions, and opportunities," *IEEE Commun. Surv. Tut.*, vol. 21, no. 3, pp. 2419–2465, Jul.–Sep. 2019.

[3] Z. Hong, H. Huang, S. Guo, W. Chen, and Z. Zheng, "QoS-aware cooperative computation offloading for robot swarms in cloud robotics," *IEEE Trans. Veh. Technol.*, vol. 68, no. 4, pp. 4027–4041, Apr. 2019.

[4] A. K. Sangaiah, D. V. Medhane, T. Han, M. S. Hossain, and G. Muhammad, "Enforcing position-based confidentiality with machine learning paradigm through mobile edge computing in real-time industrial informatics," *IEEE Trans. Ind. Informat.*, vol. 15, no. 7, pp. 4189–4196, Jul. 2019.

[5] Y. Wang, Z. Su, N. Zhang, and A. Benslimane, "Learning in the air: Secure federated learning for UAV-assisted crowdsensing," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1055–1069, Apr.–Jun. 2021.

[6] W. Chen, B. Liu, H. Huang, S. Guo, and Z. Zheng, "When UAV swarm meets edge-cloud computing: The QoS perspective," *IEEE Netw.*, vol. 33, no. 2, pp. 36–43, Mar./Apr. 2019.

[7] X. Qiu, W. Zhang, W. Chen, and Z. Zheng, "Distributed and collective deep reinforcement learning for computation offloading: A practical perspective," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 5, pp. 1085–1101, May 2021.

[8] F. Guo, H. Zhang, H. Ji, X. Li, and V. C. Leung, "Joint trajectory and computation offloading optimization for UAV-assisted MEC with noma," in *Proc. IEEE INFOCOM IEEE Conf. Comput. Commun. Workshops*, 2019, pp. 1–6.

[9] N. Dey and S. Tamane, *Big Data Analytics for Smart and Connected Cities*, Hershey, PA, USA: IGI Global, 2018.

[10] M.-A. Messous, H. Sedjelmaci, N. Houari, and S.-M. Senouci, "Computation offloading game for an UAV network in mobile edge computing," in *Proc. IEEE Int. Conf. Commun.*, 2017, pp. 1–6.

[11] X. Cao, J. Xu, and R. Zhang, "Mobile edge computing for cellular-connected UAV: Computation offloading and trajectory optimization," in *Proc. IEEE 19th Int. Workshop Signal Process. Adv. Wireless Commun.*, 2018, pp. 1–5.

[12] H. Mei, K. Yang, Q. Liu, and K. Wang, "Joint trajectory-resource optimization in UAV-enabled edge-cloud system with virtualized mobile clone," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 5906–5921, Jul. 2020.

[13] J. Zhang *et al.*, "Stochastic computation offloading and trajectory scheduling for UAV-assisted mobile edge computing," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 3688–3699, Apr. 2019.

[14] A. K. Sangaiah, J. S. Ramamoorthi, J. J. Rodrigues, M. A. Rahman, G. Muhammad, and M. Alrashoud, "LACCVoV: Linear adaptive congestion control with optimization of data dissemination model in vehicle-to-vehicle communication," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 8, pp. 5319–5328, Aug. 2021.

[15] A. K. Sangaiah *et al.*, "Energy-aware geographic routing for real-time workforce monitoring in industrial informatics," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9753–9762, Jun. 2021.

[16] C. H. Liu, Z. Chen, J. Tang, J. Xu, and C. Piao, "Energy-efficient UAV control for effective and fair communication coverage: A deep reinforcement learning approach," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 9, pp. 2059–2070, Sep. 2018.

[17] S. Fu *et al.*, "Energy-efficient UAV enabled data collection via wireless charging: A reinforcement learning approach," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 10209–10219, Jun. 2021.

[18] L. Busoniu, R. Babuska, and B. De Schutter, "A comprehensive survey of multiagent reinforcement learning," *IEEE Trans. Syst., Man, Cybern., Part C.*, vol. 38, no. 2, pp. 156–172, Mar. 2008.

[19] Z. Xiong, Y. Zhang, D. Niyato, R. Deng, P. Wang, and L.-C. Wang, "Deep reinforcement learning for mobile 5G and beyond: Fundamentals, applications, and challenges," *IEEE Veh. Technol. Mag.*, vol. 14, no. 2, pp. 44–52, Jun. 2019.

[20] Q. Long, Z. Zhou, A. Gupta, F. Fang, Y. Wu, and X. Wang, "Evolutionary population curriculum for scaling multi-agent reinforcement learning," in *Proc. 8th Int. Conf. Learn. Representations*, Addis Ababa, Ethiopia, 2020. [Online]. Available: https://openreview.net/forum?id=SJxbHkrKDH

[21] S. Iqbal and F. Sha, "Actor-attention-critic for multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 2961–2970.

[22] C. Tao, K. Zhu, B. Chen, and Y. Zhao, "UAV-assisted ground signal map construction based on 3-D spatial correlation," in *Proc. GLOBECOM IEEE Glob. Commun. Conf.*, 2020, pp. 1–5.

[23] Z. Zhou, J. Feng, B. Ai, and M. Guizani, "Energy-efficient mobile crowd sensing based on unmanned aerial vehicles," in *Proc. IEEE Glob. Commun. Conf.*, 2018, pp. 1–6.

[24] T. Mitsuhashi, Y. Kawamoto, and N. Kato, "Controlling UAV for maximizing the number of receiver vehicles in intelligent transportation systems," in *Proc. IEEE Glob. Commun. Conf.*, 2019, pp. 1–6.

[25] Z. Zhou *et al.*, "When mobile crowd sensing meets UAV: Energy-efficient task assignment and route planning," *IEEE Trans. Commun.*, vol. 66, no. 11, pp. 5526–5538, Nov. 2018.

[26] B. Wang, Y. Sun, D. Liu, H. M. Nguyen, and T. Q. Duong, "Social-aware UAV-assisted mobile crowd sensing in stochastic and dynamic environments for disaster relief networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 1, pp. 1070–1074, Jan. 2020.

[27] R. Valentino, W.-S. Jung, and Y.-B. Ko, "A design and simulation of the opportunistic computation offloading with learning-based prediction for unmanned aerial vehicle (UAV) clustering networks," *Sensors*, vol. 18, no. 11, 2018, Art. no. 3751.

[28] S. Zhu, L. Gui, J. Chen, Q. Zhang, and N. Zhang, "Cooperative computation offloading for UAVs: A joint radio and computing resource allocation approach," in *Proc. IEEE Int. Conf. Edge Comput.*, 2018, pp. 74–79.

[29] Z. Kaleem *et al.*, "UAV-empowered disaster-resilient edge architecture for delay-sensitive communication," *IEEE Netw.*, vol. 33, no. 6, pp. 124–132, Nov./Dec. 2019.

[30] Y. Wang, K. Wang, H. Huang, T. Miyazaki, and S. Guo, "Traffic and computation co-offloading with reinforcement learning in fog computing for industrial applications," *IEEE Trans. Ind. Informat.*, vol. 15, no. 2, pp. 976–986, Feb. 2019.

[31] C. H. Liu, Z. Chen, and Y. Zhan, "Energy-efficient distributed mobile crowd sensing: A deep learning approach," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 6, pp. 1262–1276, Jun. 2019.

[32] Y. Zhan, Y. Xia, J. Zhang, T. Li, and Y. Wang, "An incentive mechanism design for mobile crowdsensing with demand uncertainties," *Inf. Sci.*, vol. 528, pp. 1–16, 2020.

[33] C. Piao and C. H. Liu, "Energy-efficient mobile crowdsensing by unmanned vehicles: A sequential deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6312–6324, Jul. 2020.

[34] X. Tao and A. S. Hafid, "Deepsensing: A novel mobile crowdsensing framework with double deep Q-network and prioritized experience replay," *IEEE Internet Things J.*, vol. 7, no. 12, pp. 11 547–11 558, Dec. 2020.

[35] C. H. Liu, Z. Dai, H. Yang, and J. Tang, "Multi-task-oriented vehicular crowdsensing: A deep learning approach," in *Proc. IEEE INFOCOM IEEE Conf. Comput. Commun.*, 2020, pp. 1123–1132.

[36] W. Xu *et al.*, "Autonomous rate control for mobile Internet of Things: A deep reinforcement learning approach," in *Proc. IEEE 92nd Veh. Technol. Conf.*, 2020, pp. 1–6.

[37] B. Zhang, C. H. Liu, J. Tang, Z. Xu, J. Ma, and W. Wang, "Learning-based energy-efficient data collection by unmanned vehicles in smart cities," *IEEE Trans. Ind. Informat.*, vol. 14, no. 4, pp. 1666–1676, Apr. 2018.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on March 29,2024 at 06:52:56 UTC from IEEE Xplore. Restrictions apply.

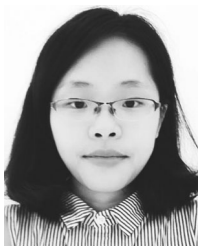
[38] C. H. Liu, Z. Dai, Y. Zhao, J. Crowcroft, D. Wu, and K. K. Leung, "Distributed and energy-efficient mobile crowdsensing with charging stations by deep reinforcement learning," *IEEE Trans. Mobile Comput.*, vol. 20, no. 1, pp. 130–146, Jan. 2021.
[39] Z. Li, M. Xu, J. Nie, J. Kang, W. Chen, and S. Xie, "NOMA-enabled cooperative computation offloading for blockchain-empowered Internet of Things: A learning approach," *IEEE Internet Things J.*, vol. 8, no. 4, pp. 2364–2378, Feb. 2021.
[40] J. Heydari, V. Ganapathy, and M. Shah, "Dynamic task offloading in multi-agent mobile edge computing networks," in *Proc. IEEE Glob. Commun. Conf.*, 2019, pp. 1–6.
[41] Y. Zhang, B. Di, Z. Zheng, J. Lin, and L. Song, "Joint data offloading and resource allocation for multi-cloud heterogeneous mobile edge computing using multi-agent reinforcement learning," in *Proc. IEEE Glob. Commun. Conf.*, 2019, pp. 1–6.
[42] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Trans. Netw.*, vol. 24, no. 5, pp. 2795–2808, Oct. 2016.
[43] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6382–6393.
[44] J. Hu, H. Zhang, L. Song, R. Schober, and H. V. Poor, "Cooperative internet of UAVs: Distributed trajectory design by multi-agent deep reinforcement learning," *IEEE Trans. Commun.*, vol. 68, no. 11, pp. 6807–6821, Nov. 2020.
[45] A. Vaswani *et al.*, "Attention is all you need," *Adv. Neural Inf. Process. Syst.*, pp. 5998–6008, 2017.
[46] T. P. Lillicrap *et al.*, "Continuous control with deep reinforcement learning," in *Proc. 4th Int. Conf. Learn. Representations*, San Juan, Puerto Rico, 2016. [Online]. Available: http://arxiv.org/abs/1509.02971
[47] J. Jiang, X. Zhang, J. Yuan, K. Tang, and X. Zhang, "Extendable flight system for commercial UAVs on ros," in *Proc. 37th Chin. Control Conf.*, 2018, pp. 1–5.
[48] L. Ferranti, L. Bonati, S. D'Oro, and T. Melodia, "Skycell: A prototyping platform for 5G aerial base stations," in *Proc. IEEE 21st Int. Symp. World Wireless, Mobile Multimedia Netw.*, 2020, pp. 329–334.



Ting Cai (Graduate Student Member, IEEE) is currently working toward the Ph.D. degree with the School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou, China. From 2012 to 2018, she was a Lecturer with the College of Mobile Telecom, Chongqing University of Posts and Telecom, Chongqing, China. Her current research interests include blockchain, deep reinforcement learning, Internet of Things security, and edge/cloud computing.



Zhihua Yang received the B.S. degree in 2019 from Sun Yat-sen University, Guangzhou, China, where he is currently working toward the M.S. degree with the School of Computer Science and Engineering. His current research interests include mobile crowdsensing, edge computing, and deep reinforcement learning.



Yufei Chen received the B.S. degree in 2019 from Sun Yat-sen University, Guangzhou, China, where she is currently working toward the M.S. degree with the School of Computer Science and Engineering. Her current research interests include blockchain, Internet of Things, edge computing, and auction theory.



Wuhui Chen (Member, IEEE) received the bachelor's degree from Northeast University, Shengyang, China, in 2008, and the master's and Ph.D. degrees from the University of Aizu, Aizuwakamatsu, Japan, in 2011 and 2014, respectively. From 2014 to 2016, he was a Research Fellow with the Japan Society for the Promotion of Science, Tokyo, Japan. From 2016 to 2017, he was a Researcher with the University of Aizu. He is currently an Associate Professor with Sun Yat-sen University, Guangzhou, China. His current research interests include edge/cloud computing, cloud robotics, and blockchain.



Zibin Zheng (Senior Member, IEEE) received the Ph.D. degree from The Chinese University of Hong Kong, Hong Kong, in 2011. He is currently a Professor with the School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou, China. He has authored or coauthored more than 150 international journal and conference papers, including three ESI highly cited papers. According to Google Scholar, his papers have more than 13,590 citations, with an H-index of 54. His research interests include blockchain, smart contract, services computing, and software reliability.



Yang Yu (Member, IEEE) received the bachelor's and master's degrees in computer science from the Huazhong University of Science and Technology, Wuhan, China, in 1998 and 1991, respectively, and the Ph.D. degree in computer science from Sun Yat-sen University (SYSU), Guangzhou, China, in 2007. From 1991 to 2002, he was a Senior Engineer and CTO with a software company. In 2003, he became a Vice Professor with the School of Information Science and Technology, SYSU, where he has been a Full Professor with the School of Data and Computer Science since 2011. He has authored or coauthored more than 60 papers and has been a reviewer for several prestigious international conferences and journals. His research interests include workflow, service computing, cloud computing, and software engineering. He is a Senior Member of CCF and a Member of ACM.



Hong-Ning Dai (Senior Member, IEEE) received the Ph.D. degree in computer science and engineering from the Department of Computer Science and Engineering, The Chinese University of Hong Kong, Hong Kong. He is currently an Associate Professor with the Department of Computing and Decision Sciences, Lingnan University, Hong Kong. His current research interests include the Internet of Things, big data, and blockchain technology. He was as Associate Editor for the IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, IEEE SYSTEMS JOURNAL, IEEE ACCESS, *Ad Hoc Networks*, and *Connection Science*. He is also a Senior Member of the Association for Computing Machinery (ACM).

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on March 29,2024 at 06:52:56 UTC from IEEE Xplore. Restrictions apply.